

AI Transformation with the ATANDA Command Centre

A Practitioner's Guide — SAFe-Aligned, Reality-Bound

Version 1.1 · May 2026 — revised after deep code audit

About this document

This guide is grounded in **what the ATANDA Command Centre actually ships today**, not what its parent literature aspires to. The platform is, in one line, a single-creator SaaS that walks one user through a deterministic prompt → SPC → PDD → certified MVP-PDD pipeline (engines F1–F7), with an optional F8 Code DJ that scaffolds runnable code from the certified spec.

Around that core loop the platform provides: Clerk authentication, four Stripe-billed tiers (Explorer, Practitioner, Architect, Institution) plus a Team Lite per-seat plan, per-tier daily rate limits on each engine, a monthly LLM cost cap with admin override, optional team-seat organisations, an Ingestion engine that swallows a prior IPDD, an optional Cartridge bundle, a Sphinx Marketplace publish hook, an Activity Log, and a `/me/costs` dashboard.

This document maps that real surface onto the Scaled Agile Framework (SAFe) — PI Planning, iteration cadence, System Demo, and Inspect & Adapt — and gives two concrete guides:

- **Part 1** is a narrative of one ART running one PI through the platform.
- **Part 2A** is a step-by-step guide for **Greenfield** AI transformation projects.
- **Part 2B** is the same guide adapted for **Brownfield** projects with an existing spec or codebase.

Throughout, every step references a real engine, route, or schema on the platform. Where SAFe asks for capability the platform does not have, the guide says so plainly and recommends the off-platform tool that fills the gap.

PART 1 — Narrative

Helix Logistics runs PI-26.Q3 on the Command Centre

Context. Helix Logistics is a mid-sized third-party logistics operator. Their dock-door exception triage today is a manual queue: a shift lead reads each carrier exception ticket, classifies it, and routes it to the right desk. Average handling time is six minutes per ticket, ~1,400 tickets per day. The ART has decided this is the workflow for their next PI: build an AI-assisted classifier and routing assistant that the shift lead reviews, not one that acts autonomously. The Release Train Engineer (Maya) and the Lead Solution Architect (Devon) own delivery; four feature teams will build alongside.

Subscription posture. Devon, the architect, pays for the **Architect** tier (\$199/month). The ART buys four **Team Lite** seats at the per-seat price; those seats elevate to ARCHITECT-equivalent rate limits and gain 2 F8 runs per day each. They do not buy the **Institution** tier yet — they do not need unlimited F8, and they want to see the cost shape of one PI before committing.

Pre-PI Planning (the week before PI Planning). Devon opens the Command Centre and creates a new session named `pi-26q3-dock-exception-triage`. He pastes the rough business hypothesis into the F1 workspace: *"We can cut dock-door exception triage handling time in half by surfacing a top-3 classification and a recommended desk for each ticket, with the shift lead always making the final call."* F1 returns a 7-pillar diagnostic. The ROLE pillar comes back weak — *"too many implicit actors; whose AI is this, the shift lead's or the carrier desk's?"* Devon iterates twice and lands a tightened version. The diagnostic, the iterations, and the final score are persisted under `harness_artifacts` for the session; everything is visible in the Activity Log.

He then runs **F2 (Build Atomic Prompt)** against the tightened brief. F2 produces the canonical atomic prompt — the unit the rest of the HARNESS will operate on. JCSE score (Junglenomics Composite Score Estimate, a 0–50 rubric baked into every scoring engine's system prompt) comes back at 47/50. Good enough.

PI Planning, Day 1. Devon presents the F2 atomic prompt as the **Solution Intent draft** in PI Planning. The four feature teams break it down into features and stories on the SAFe board. Maya tracks dependencies. One feature team raises a flag: *"We don't know if our exception data is clean enough."* Devon notes this as a Phase 2 readiness risk — this is where the platform leaves the SAFe ART on its own; there is no readiness-index engine, the team will assess manually using their own data-quality tooling.

PI Planning, Day 2. Devon runs **F3 (CELL — Micro Agent Birth Package)** on the atomic prompt. F3 takes ~60–90 seconds to return (it is a long-running request/response, not a streamed SSE engine — the only SSE endpoint on the platform today is `/harness/escalations/stream`). The CELL package defines the agent's birth conditions using archetype-tagged pillars (SPHINX / SPARTAN / ADA / SOLVA — these archetype labels are literally in the F3 prompt, not just marketing). Two feature teams take dependencies on the output contract; this becomes a hard interface for them.

He then runs **F4 (Micro PDD)** — a one-page PDD that crystallises the CELL into a development-ready spec. The Micro PDD is exported as an artifact and pinned to the ART's Confluence (the platform does not have its own document-sharing surface; export-and-pin is the pattern).

Iteration 1. The four feature teams build. Devon runs **F5 (Build SPC)** in parallel. F5 walks the canonical FORGE 7-step Q&A script (the seven step names are hard-coded into the F5 prompt) and produces a 15-section SPC with HIVE 14-D scoring across dimensions including Ethics/GRO and Token Optimization/ZPOS. JCSE comes back 41/50; the CONSTRAINT pillar is flagged for under-specification of failure modes. Devon iterates twice and lands at 46/50, cert class GOLD (the SILVER/GOLD/PLATINUM cert tiers are real, enumerated in the F5 finaliser prompt). Devon (Architect tier) has unlimited F5 runs per day; he uses 3.

Iteration 2. Devon runs **F6 (Draft ATLAS PDD)** — the 4-Part PDD that the feature teams will use as their authoritative spec. He also runs **F6-VDJ** to get the VIBE DJ recommendation: which coding tool (Cursor / Lovable / v0 / etc.) is the best primary conductor for this workload. F6-VDJ recommends Cursor at 88/100 because the team already has a Cursor convention. Devon attaches the recommendation as a System Architect note in the SAFe board.

During Iteration 2 the team hits the monthly cost cap warning at 70% of the Architect tier's \$250/month cap. Devon checks `/me/costs`. F5 and F6 dominate the cost — predictable, those are the long-context engines. He decides to pace the remaining iterations and not pre-run F7 until the team is ready.

Iteration 3 + System Demo. Devon runs **F7 (Convert to MVP PDD)** which executes the seven-step SPARTAN pipeline (SCAN → PROFILE → ASSESS → REDUCE → TRANSFORM → ZPOS → PACKAGE — all in F7's engine code) and emits an MVP-PDD with a SPARTAN cert ID (format `SPARTAN-YYYYMMDD-shortuuid`). The cert artifact has a verify page at `/verify?artifactId=...`. If Devon also publishes to Sphinx (requires `SPHINX_BASE_URL` to be configured), the verify URL becomes externally public. The MVP-PDD is the demo

artifact for the System Demo. The shift lead persona runs through three exception tickets against the not-yet-built model; the team aligns on whether the certified contract matches what the shift lead actually needs.

Iteration 4. One feature team uses **F8 Code DJ** to scaffold the model-serving wrapper from the certified MVP-PDD. F8 produces a runnable codebase that the team takes into Cursor for completion. Two Team Lite seats burn their daily F8 quota (2/day each); the architect's Architect tier seat also burns one F8 call. The platform's per-engine `harness_engine_runs` table records cost per run; this lands in `/me/costs`.

PFP run before merge. After F8 scaffolds the code (PFP runs **post-F8**, not pre-F8 — it checks the scaffold against the certified spec, it does not auto-gate F8 itself), Devon runs the **PFP (Promptware Failure Predictor)**. PFP analyses the F8 output for drift from the F7-certified spec; the verdict and finding counts are recomputed server-side from `findings` so the model's self-reported numbers cannot bypass the check. Two findings come back, one flagged HIGH (the output contract is missing the `confidence_score` field that the SPC promised). The team enforces the merge block as a manual policy — the platform reports drift, humans gate the PR. The fix lands before merge.

Inspect & Adapt. At I&A, the ART pulls the Activity Log filtered to the PI's session ID. The log is the audit trail: every engine run, who ran it, what the input was, what the output JCSE score was, what the cost was. The retrospective surfaces two things: (a) F5 iterations are the most expensive learning cycle — invest more time in F3/F4 to reduce F5 rework next PI; (b) the team did not use the **Sphinx Marketplace** publish — they could have published the certified spec for the wider organisation to fork. Action item for PI N+1.

Cartridge decision. At the end of the PI the team has a certified, code-scaffolded, drift-checked deliverable. They consider the **Cartridge** (\$499.99 one-time per project) which bundles the certified artifacts as a packaged hand-off. They buy it for this PI because their Ops VP wants the bundle for the compliance file.

What the platform did NOT do in this PI. It did not score Helix's organisational AI readiness. It did not assess the carrier-exception dataset for completeness. It did not provide a Safety Mode runtime for the deployed classifier. It did not capture telemetry from the live shift-lead UI. It did not generate compliance documentation against any specific regulation. All of those were owned off-platform by the ART, the SRE team, the data team, and Compliance. The Command Centre's contribution was *the certified spec, the scaffolded code, the drift check, and the audit trail of how those were produced*.

PART 2A — Greenfield Step-by-Step

A SAFe-aligned, Command Centre-grounded guide

This part assumes a new AI workflow with no prior spec, no prior code, and a fresh ART forming around it. The cadence below maps to one Program Increment (PI). Repeat for each PI.

Step 0 — Subscription posture (one-time, before PI 1)

Role	Tier	Why
Lead Solution Architect	Architect (\$199/mo)	Highest daily limits on F5/F6/F7; 8 F5/day, 8 F6/day, 4 F7/day.
Each feature team lead	Team Lite seat (per-seat)	Elevates to ARCHITECT-equivalent rate limits + 2 F8/day each.
Wider engineering org (read-only)	Explorer (free)	Lets them browse exemplars and run F1 diagnostics.

Role	Tier	Why
Centralised platform team (optional)	Institution (per-seat via team subs)	Only if you need unlimited F8. Note: there is no flat-rate Institution SKU — it is sold per seat via STRIPE_PRICE_TEAM_SEAT_*

Set the org's monthly cost-cap override if the default (\$250 for Architect, ~\$2000 for Institution per seat) is wrong for your workload. The admin route is PATCH /api/admin/subscribers/:userId/cost-cap.

Step 1 — Pre-PI Planning week (the architect alone)

- **Open a session** named for the PI (pi-N-<workflow-slug>). Sessions are the unit of context for the platform — every artifact in this PI lives under one session id.
- **Run F1 (Test Your Prompt)** against the business hypothesis. Iterate until the 7-pillar diagnostic scores all pillars green and JCSE $\geq 45/50$. Budget 3–5 F1 iterations.
- **Run F2 (Build Atomic Prompt)** on the tightened hypothesis. You now have the canonical atomic prompt for the PI. This is your **Solution Intent draft**.
- **Do NOT run F5/F6/F7 yet.** They are expensive; you want feature-team input first.

Step 2 — PI Planning Day 1 (architect + ART)

- **Present the F2 atomic prompt as the Solution Intent.** This replaces the usual whiteboard-and-sticky-notes solution intent with a structured, scored artifact.
- **Feature teams break down features and stories** against the atomic prompt's pillars. ROLE, INSTRUCTION, and FORMAT pillars are the natural seams for feature decomposition.
- **Identify off-platform readiness gaps.** The platform does not score organisational AI readiness or data foundation quality. Surface these as PI risks on the ROAM board (Resolved / Owned / Accepted / Mitigated).

Step 3 — PI Planning Day 2 (architect)

- **Run F3 (CELL Micro Agent Birth Package).** F3 is SSE — it streams output for 60–120 seconds. The output defines the agent's input/output contract.
- **Run F4 (Micro PDD).** This is the development-ready spec the feature teams will work from.
- **Export the Micro PDD** and pin it to wherever your ART normally pins specs (Confluence, Notion, Linear). The platform does not host docs for external consumption.
- **Capture dependencies on the F3 output contract.** This is your ART-level integration contract.

Step 4 — Iterations 1–2 (feature teams + architect in parallel)

- **Feature teams build** against the Micro PDD.
- **Architect runs F5 (Build SPC)** at start of Iteration 1. F5 is the heaviest cost-per-run engine. Plan 2–3 iterations to hit JCSE $\geq 45/50$.
- **Architect runs F6 (Draft ATLAS PDD)** at start of Iteration 2. F6 produces the 4-Part PDD the teams will treat as authoritative.
- **Run F6-VDJ** to get the VIBE DJ recommendation for primary coding tool. Pin the recommendation as a System Architect note.

- **Monitor** `/me/costs`. The dashboard shows a 30-day cost chart, per-engine breakdown, and the current cap meter. **The platform does not push a warning at any threshold** — it returns 402 `COST_CAP_EXCEEDED` only when the cap is actually hit. Architects must check the dashboard themselves. Pace runs if you cross 60% of the cap before mid-PI.

Step 5 — Iteration 3, System Demo prep (architect)

- **Run F7 (Convert PDD to MVP)**. F7 walks the SPARTAN seven-step pipeline (SCAN→PROFILE→ASSESS→REDUCE→TRANSFORM→ZPOS→PACKAGE) and emits a SPARTAN-certified MVP-PDD. The verify page is local by default; publish to Sphinx for an externally public URL (requires `SPHINX_BASE_URL`).
- **The verification URL is the System Demo artifact**. It links the hypothesis → atomic prompt → CELL → Micro PDD → SPC → ATLAS PDD → certified MVP-PDD.
- **Walk the personas through the certified contract** at System Demo. Capture mismatches as Iteration 4 stories.

Step 6 — Iteration 4 (feature teams + architect)

- **Feature teams use F8 Code DJ** to scaffold model-serving code from the certified MVP-PDD. F8 is rate-limited per-day per-seat; plan accordingly.
- **Run PFP (Promptware Failure Predictor)** on every F8 output before merge. PFP runs **after F8** and reports drift against the F7-certified spec; its verdict is recomputed server-side from `findings` so the model's self-reported numbers cannot bypass the check. The platform does NOT auto-block F8 or auto-block your PR — your CI/team policy is the actual gate. Block merge on any HIGH finding.
- **Off-platform**: the actual model training, deployment, observability, and human-in-the-loop UI all live in your normal engineering stack. The platform's contribution stops at the certified spec + scaffolded code + drift check.

Step 7 — Inspect & Adapt (ART)

- **Pull the Activity Log** filtered to the PI's session id. This is your audit trail.
- **Pull** `/me/costs` for the architect's account. Per-engine cost breakdown shows where your iteration cycles were most expensive.
- **Two questions for the retrospective**:
 - 1 Which engine had the most rework cycles? (Usually F5 — invest earlier in F3/F4 next PI.)
 - 2 Did the certified MVP-PDD match what shipped? (Drift here means the SPC was under-specified — tighten the CONSTRAINT pillar next PI.)
- **Optional: publish to Sphinx Marketplace**. If the certified spec is reusable, publish it via `/api/integrations/sphinx/publish` (requires `SPHINX_BASE_URL` to be configured). This is how your ART contributes to the wider organisation.

Step 8 — Cartridge decision (architect + business owner)

- **At PI close**, decide whether to buy the Cartridge (\$499.99 one-time per project). The Cartridge bundles the certified artifacts for hand-off to Compliance, Ops, or an external client.
- **Buy it if**: you need a packaged audit-ready hand-off, or the spec is being sold/transferred externally.
- **Skip it if**: the artifacts live happily in your internal systems and never need a portable bundle.

Step 9 — PI N+1 ramp

- **Branch the session.** Create a new session for PI N+1 with the previous PI's MVP-PDD as the seeded context (via the Ingestion engine — feed the prior MVP-PDD as an IPDD).
 - This is the brownfield pattern applied to your own prior work, and it is how greenfield work compounds across PIs.
-

PART 2B — Brownfield Step-by-Step

When you already have a spec, a codebase, or both

Brownfield AI transformation has three flavours. Pick yours before starting:

- 1 **Doc-only brownfield.** You have a PDD, SDD, brief, or concept note. No code yet.
- 2 **Code-only brownfield.** You have a deployed AI workflow that is underperforming or failing, and no formal spec.
- 3 **Doc-and-code brownfield.** You have both, and they have drifted apart.

The platform's strongest support is for case (1). Case (2) requires you to author a Failure-DNA-Map narrative off-platform and ingest it. Case (3) is the hardest — you reconcile by treating the deployed code's *actual* behaviour as the IPDD, not the legacy doc.

Step 0 — Subscription posture

Identical to greenfield. Brownfield does not require a higher tier; if anything, brownfield uses **fewer** F5/F6 iterations because the inbound context is already structured.

However: brownfield projects often need the **Ingestion** engine, which is billed separately as a per-project line item (STRIPE_PRICE_INGESTION_PROJECT). Budget for one Ingestion per brownfield PI.

Step 1 — Pre-PI Planning week (architect alone)

- **Prepare the IPDD.** Whatever you have — the legacy PDD, the SDD, the concept note, the runbook, the post-mortem of the last failed attempt — assemble it into one document. This becomes the **IPDD** (Ingestion Product Design Document, the platform's formal input name).
- **For code-only brownfield:** the architect writes a 2–4 page narrative of what the deployed workflow *actually does today*, what it gets wrong, and the hypothesised root cause. This narrative IS the IPDD. Do not skip it.
- **Open a session** named for the PI. Set `origin = 'ingested'` (the platform marks ingested sessions with one of three values: `manual`, `ingested`, `cartridge`). Note: ingested sessions are conventionally referred to as producing **PWDDs**, but F7's stored `artifactType` is still `MVP_PDD` — the PWDD distinction is a user-facing terminology layer, not a database enum value today.
- **Run the Ingestion engine** on the IPDD. The Ingestion engine normalises the input into the seed context for F1.
- **Consider an Advanced Cartridge (\$499.99 one-time per project) instead of, or alongside, plain Ingestion.** The Cartridge mechanism is meaningfully more powerful: cartridge files (PDFs, DOCX, JSON specs, prompt inventories) are uploaded once and then **prepended to every subsequent engine call within that session**, fenced with `=== CARTRIDGE CONTEXT (authoritative · do not contradict) ===`. Every F1 diagnostic, every F5 SPC, every F7 SPARTAN compression runs with the full legacy context in scope. For brownfield work with a deep legacy surface, this is the highest-leverage spend on the platform.

Step 2 — Pre-PI Planning, narrowing scope

Brownfield projects almost always have scope ambition that exceeds one PI. Resist it.

- **Run F1 (Test Your Prompt)** against the ingested context, but with a **narrowing prompt**: *"Of the workflows in this IPDD, which single workflow has (a) the highest measurability, (b) the lowest workforce resistance, and (c) C-suite visibility within 90 days?"*
- F1's diagnostic will surface a candidate. Iterate until you have one workflow.
- **Run F2 (Build Atomic Prompt)** on that single workflow. This is your **Solution Intent draft** — a narrowed version of the inbound IPDD's full ambition.

Step 3 — PI Planning Day 1 (architect + ART)

- **Present F2 as the narrowed Solution Intent.** Explicitly contrast it against the legacy IPDD's full scope; record the deferred scope as a backlog for future PIs.
- **Identify integration constraints.** Brownfield deployments live next to legacy systems. Surface every integration touch-point as a PI risk on the ROAM board.
- **Capacity adjustment.** Brownfield velocity is typically 60–70% of greenfield for the same notional scope. Plan accordingly.

Step 4 — PI Planning Day 2 (architect)

- **Run F3 (CELL).** The CELL's input contract **MUST** be the contract the legacy system already emits — do not invent a clean contract that requires legacy-side change in this PI.
- **Run F4 (Micro PDD).** Annotate every interface with "legacy-bound" or "greenfield-bound" so feature teams know which contracts they own and which they inherit.

Step 5 — Iterations 1–2 (feature teams + architect)

- **Run F5 (Build SPC)** with explicit CONSTRAINT pillar entries for: legacy data quirks, legacy auth model, legacy rate limits, and any deprecated fields the legacy system still emits.
- **Run F6 (ATLAS PDD)** with a 4th-part GOVERNANCE section that names the parity gate: the new AI workflow must match or beat the legacy baseline on the chosen measurability metric.
- **Run F6-VDJ** for the coding tool recommendation. For brownfield, F6-VDJ often recommends an IDE-extension tool (Cursor) over a from-scratch builder (Lovable), because the work is integration-heavy rather than greenfield-heavy.

Step 6 — Iteration 3, System Demo prep

- **Run F7 (Convert to MVP — outputs a PWDD for ingested sessions).** The PWDD is the certified narrowed-and-integrated spec, with a public verification URL.
- **Demo against the legacy baseline.** The System Demo's measurability question is not "does it work?" — it is "does it beat the baseline on the agreed metric?"
- **If the PWDD does not credibly beat baseline:** stop. Do not let an under-performing AI workflow ship as a brownfield replacement; the cultural cost of a visible regression is high. Loop back to Step 2 and re-narrow.

Step 7 — Iteration 4 (feature teams + architect)

- **F8 Code DJ scaffolds the new workflow's serving layer**, not the integration layer. The integration into the legacy system is hand-written and owned by the feature team — the platform does not understand your legacy stack.
- **Run PFP on F8 output before merge**. Same gate as greenfield. Block merge on HIGH findings.
- **Add a parity test alongside the unit test suite**. The parity test replays the last 30 days of legacy traffic against the new workflow and compares the agreed metric. The platform does not run this test — it is an off-platform CI job.

Step 8 — Inspect & Adapt (ART)

- **Pull the Activity Log + /me/costs** as in greenfield.
- **Two brownfield-specific retrospective questions:**
 - 1 Where did our F5/F6 specs assume a clean greenfield contract and have to be re-iterated to accept legacy reality?
 - 2 What deferred scope from the original IPDD is now the strongest candidate for PI N+1?

Step 9 — Cartridge decision

- **Brownfield projects buy the Cartridge more often than greenfield** because the audit trail of *what changed vs the legacy baseline* is exactly what Compliance and Ops want.

Step 10 — PI N+1 ramp

- **Feed the prior PI's PWDD as the IPDD for PI N+1**. Brownfield compounds via successive narrowed-and-integrated PWDDs — each PI peels off one more workflow from the deferred scope backlog.

What this guide deliberately does NOT cover

The Command Centre as it exists today does not provide the following. If your SAFe ART needs them, source them from elsewhere — do not assume the platform will:

- **Organisational AI readiness scoring**. Use your own change-management framework.
- **Data foundation quality assessment**. Use your data team's normal tooling (Great Expectations, dbt tests, etc.).
- **Vendor due-diligence / demo-vs-reality scoring**. Run your normal procurement process.
- **Runtime safety-mode state machine for deployed AI**. Use an AI observability product (Arize, Fiddler, Datadog AI Observability) for production behaviour control.
- **Live telemetry from deployed AI systems**. Same — the platform's telemetry is on its own engine runs, not on your deployed model.
- **Regulatory compliance matrices** (HIPAA / SR 11-7 / EU AI Act / NIST AI RMF / ISO 42001). The PDD section names them; maintaining a living compliance state is a regtech product.
- **Multi-role checkpoint approval workflow with e-signature**. SAFe ART roles (RTE, PO, Architect, Business Owner) sign off in your normal SAFe tooling. The platform tracks one role: subscriber.
- **Adversarial red-team simulation, OWASP LLM Top 10 enumeration**. PFP detects drift on F8 output. It does not red-team your deployed system.

- **Re-certification cron jobs.** The platform has a daily rate-limit reset cron and a weekly digest cron. It does not have a 90-day re-certification cron for prior certified specs.

These gaps are not bugs — they are scope. The Command Centre's scope is *the certified spec, the scaffolded code, the drift check, and the audit trail of how those were produced*. SAFe gives you the cadence and the roles; the platform gives you the artifact pipeline; your existing tooling stack does everything else.

Quick reference: SAFe cadence ↔ Command Centre engines

SAFe event	Engine(s) used	Cost band
Pre-PI prep (architect alone)	F1, F2	Low
PI Planning Day 1	F2 output presented	Zero
PI Planning Day 2	F3, F4	Medium (F3 is long-running, ~60–90s, not SSE)
Iteration 1	F5 (2–3 iterations)	High (FORGE 7-step Q&A + HIVE 14-D scoring)
Iteration 2	F6, F6-VDJ	Medium
Iteration 3 (System Demo prep)	F7	High (SPARTAN 7-step pipeline, long-running, not SSE)
Iteration 4	F8 Code DJ, then PFP	Medium per F8 call; PFP runs post-F8
Inspect & Adapt	Activity Log + /me/costs	Zero
Brownfield PI start (recommended)	Advanced Cartridge upload	\$499.99 one-time — highest-leverage spend; prepends legacy context to every subsequent engine call
End of PI	Cartridge bundle hand-off (optional)	\$499.99 one-time
PI N+1 ramp	Ingestion + F1	Low (+ Ingestion line item)

End of guide.

Generated against ATANDA Command Centre platform state, May 2026.

Appendix — v1.1 corrections vs v1.0

This version was revised after a deep audit of the engine system prompts and route handlers. Material changes from v1.0:

- 1 **SSE claim corrected.** v1.0 said F3 and F7 stream via SSE. They do not. Both are long-running request/response engines. The only SSE endpoint on the platform today is /harness/escalations/stream.

- 2 **PFP positioning corrected.** v1.0 implied PFP gates F8 pre-merge. PFP actually runs *after* F8 to check the scaffold against the F7-certified spec. The platform does not auto-block — humans gate the PR.
- 3 **Cartridge mechanism strengthened.** v1.0 underplayed the Cartridge. The mechanism is real and powerful: cartridge files are uploaded once and then prepended to every subsequent engine call within that session, fenced as `=== CARTRIDGE CONTEXT (authoritative · do not contradict) ===`. For brownfield, this is the highest-leverage spend at \$499.99/project.
- 4 **HIVE 14-D / GRO / ZPOS+5 positioning corrected.** These are scoring conventions baked into the engine system prompts (the LLM is asked to apply them), not separate engines or runtime state machines. JCSE itself is the "Junglenomics Composite Score Estimate" defined in the F1/F2/F5 prompts.
- 5 **Persona archetypes clarified.** SPHINX, SPARTAN, ADA, SOLVA (and others) appear in engine prompts as pillar archetype tags and in the exemplar library as fork-able starting points. They are NOT runtime multi-agent panels.
- 6 **Cert tier language pinned to what the F5 prompt actually enumerates: SILVER / GOLD / PLATINUM.** "FORGE Stage 7 Platinum Tier" is the right family of label but the "Stage 7" specificity isn't in code.
- 7 **Institution tier shape clarified.** Institution is sold per-seat via `STRIPE_PRICE_TEAM_SEAT_*`. No flat-rate Institution SKU exists.
- 8 **Cost-cap warning behaviour clarified.** The platform 402s only when the cap is exceeded — there is no push notification at 70% or any other threshold.
- 9 **PWDD terminology clarified.** Sessions with `origin='ingested'` are referred to as producing PWDDs, but the stored `artifactType` is still `MVP_PDD` — PWDD is a terminology layer, not a database enum value today.
- 10 **SPARTAN cert ID format documented.** Format is `SPARTAN-YYYYMMDD-shortuuid`; the verify page is local at `/verify?artifactId=...` and only becomes externally public when published to Sphinx (`SPHINX_BASE_URL` required).